HEXAGON

Interfaces for BendingStudio XT

Manual

Hexagon AICON ETALON GmbH
Biberweg 30 C
D-38114 Braunschweig
Germany

tel. +49 (0) 5 31 58 000 58
support.tube.mi@hexagon.com
www.hexagon.com

Table of contents

1 About this manual 4

1.1 License terms 5

2 Automation interface 7

2.1 PLC hardware 9

2.1.1 Overview 10

2.1.2 Control center 13

2.1.2.1 Login 13

2.1.2.2 Dashboard 14

2.1.2.3 Last commands 14

2.1.2.4 Connect to BendingStudio XT 14

2.1.2.5 Assign the ID 16

2.1.2.6 PIN view 17

2.1.3 Setup 19

2.1.3.1 Configure the network adapter 19

2.1.3.2 Connection settings 20

2.1.4 Example: UR Cobot 22

2.1.5 PLC box 23

2.2 XML interface 24

2.2.1 Queries 25

2.2.1.1 Coding description 25

2.2.1.2 Code samples 26

2.2.2 Commands 27

2.2.2.1 Coding description 27

2.2.2.2 Code samples 30

2.2.3 Return 33

2.2.3.1 Coding description 33

2.2.3.2 ReturnCode, ErrorCode and WarningCode 34

2.2.3.3 Payload - Object 38

2.2.3.3.1 Measurement result 39

2.2.3.3.2 Correction result 41

2.2.3.3.3 Report 43

2.2.3.4 Examples 43

2.2.3.4.1 Examples for answering queries 43
2.2.3.4.2 Example for answering commands 46
2.2.3.5 Broadcasts 49
2.2.3.5.1 List of broadcasts 50

2.3 Automation Tester 51
2.3.1 How to use the tester 51

2.4 Tips and tricks for the start 53
2.4.1 Result evaluation 53
2.4.2 Automatic log-in 55

1 About this manual

This user manual is part of the product and the overall documentation of the TubeInspect.

Keep this information in a safe place. Carefully read and follow the instructions given in the manual. Hand the manual and any other product related documentation to each person using the product. Read and follow all hazard warnings and safety measures. Failing to do so will result in voiding the warranty and losing the entitlement to warranty claims. Service and repair work of the system must only be carried out by an authorised service technician or a staff member of Hexagon AICON ETALON GmbH.

This manual including all its parts is copyrighted. Copies thereof, even in the form of extracts, may only be made with express permission of Hexagon AICON ETALON GmbH. This manual has been carefully prepared. However, we cannot guarantee the correctness of the contents. Since mistakes cannot be completely avoided despite all efforts, we are grateful for each tip in this respect.

Version 07/2025, Subject to alterations.

Copyright 2025 by Hexagon AICON ETALON GmbH
Biberweg 30 C, D-38114 Braunschweig
Internet: www.hexagon.com
E-Mail: support.tube.mi@hexagon.com

In case of enquiries or queries, please always state the version or series number of the product.

1.1 License terms

License:

The buyer is allowed a single non-transferable license for the use of these programme copies and the enclosed documentation by the company Hexagon AICON ETALON GmbH only on the following terms:

The licensee is entitled to:

- make one backup copy of the software in machine-readable form, on condition that all notes, which concern protection will also be copied, as well as
- to physically transfer the software from one computer onto another, on condition that it will only be used on one of them.

The licensee is <u>not</u> entitled to:

- modify, translate, technically change, decompile, disassemble or copy (with the exception of one backup copy) the software or the enclosed documentation,
- lease, lend or transfer the license and its connected rights to a third party without the prior written permission of Hexagon AICON ETALON GmbH and
- to remove any notes concerning the protection as well as marks or labels of the software or the enclosed documentation. This license is not a sale. Property and copyrights remain in possession of Hexagon AICON ETALON GmbH.

Unauthorized copying or transferring of the software and its documentation as well as offence against the restrictions named in the contract result in the termination of the license. In this case, the company Hexagon AICON ETALON GmbH will reclaim the software, possible copies and the documentation from you. Claims that are based on subsequent damages remain untouched.

No warranty for consequential damages

Neither Hexagon AICON ETALON GmbH nor the supplier will be liable to you for any damages, including any lost profits, lost savings, or other incidental or consequential damages arising out of the use or inability to use the software even if Hexagon AICON ETALON GmbH has been advised of the possibility of such damages. In any case, Hexagon AICON ETALON GmbH only warrants for the amount, which was actually paid for the product. Claims that are based on unalienable law concerning product liability remain untouched.

The terms and conditions of the company Hexagon AICON ETALON GmbH apply.

Third party licenses

The BendingStudio XT software includes third party code (Link to terms of licenses).

2 Automation interface

The BendingStudio XT automation module is optionally available for the models TubeInspect 37 M, TubeInspect 37 XL and the HRC models of the TubeInspect P16.2 and TubeInspect P8.2 series. To use it, the BendingStudio XT AUTOMATION package is required. This documentation of the BendingStudio XT automation interface describes the interaction between the BendingStudio XT software and an external controller. BendingStudio XT acts as server, allowing a connected client to execute commands and to acquire result and correction data.

With this interface, BendingStudio XT supports robot-assisted manufacturing with the automation module. The remote access allows to trigger measurements and retrieve results. For this purpose, commands and queries are executed via an XML interface. The commands include calibrating the measuring system, triggering a measurement, requesting corrections or results and creating reports. The queries contain information about available components and inspection plans.

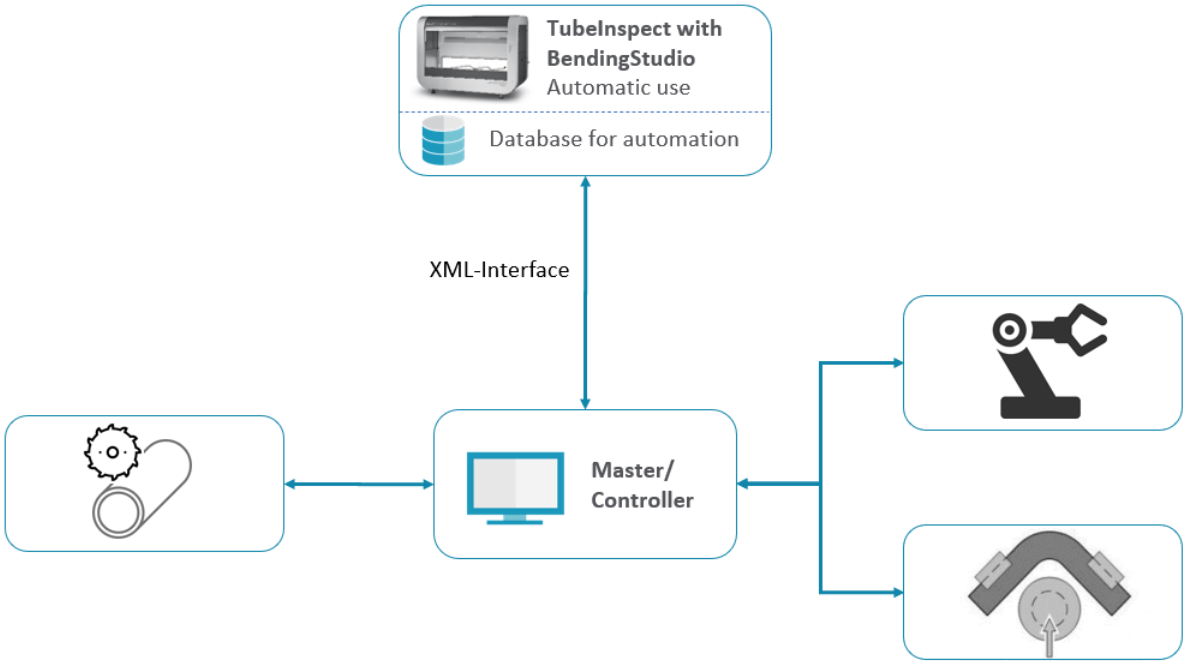


Figure 1: The automation interface of BendingStudio XT

To start the external control for BendingStudio XT, the respective settings must be defined in BendingStudio XT. In the database, inspection plans and parts must be selected and marked for automation. Subsequently, the software can be set to automatic mode to communicate with a controller via the interface. In automatic mode, the database is not updated; only when the automatic

mode is closed, a limited number of results and corrections is transferred to the source database. The existing correction values can be sent to the bending machine during normal operation, then the automation can be continued. Combining the data service with a central database is possible.

INFO Refer to the BendingStudio XT software manual for more information about how to setup a part in BendingStudio XT software to be used with automation.

The following chart shows an example of a simplified workflow for an automated production. The automation interface is initialised, available part geometries and inspection plans are queried and a calibration command is sent to the TubeInspect cell before any part is placed or held in the TubeInspect. Parts are measured and reports are created in a loop. This also contains some basic error handling.

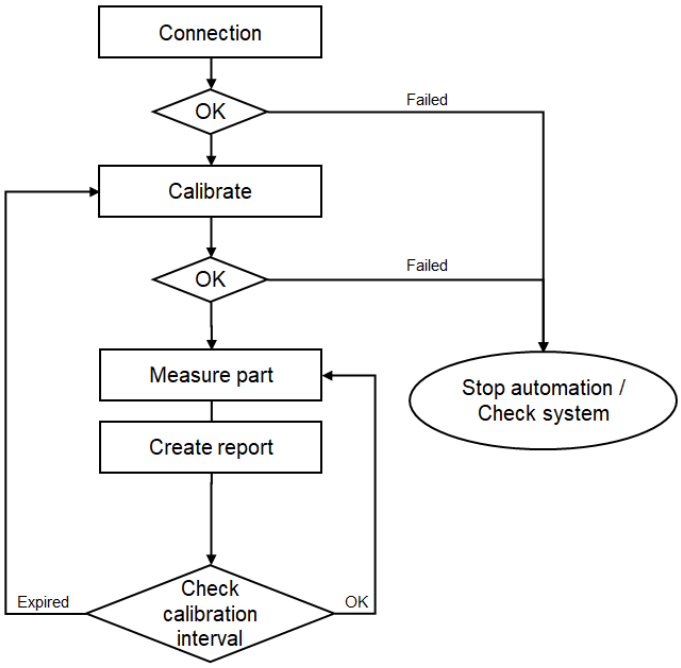


Figure 2: Example: Simplified workflow for an automatic production

Such a workflow can be realised using a dedicated PLC hardware (Programmable Logic Controller) for BendingStudio XT **or** by implementing the communication in a controller using XML to send commands to BendingStudio XT. Both, usage of PLC and XML interface are described in detail in the next sections.

2.1 PLC hardware

The PLC (Programmable Logical Controller) with digital I/O signals can directly be used to setup an automated workflow. The firmware installed on the PLC communicates directly with the BendingStudio XT automation interface triggered by the I/O signals of the PLC. An XML programming of the interface is therefore not required, the PLC automation is connected directly to the handling system (e.g. robot). The PLC is configured via a web interface.

INFO The PLC hardware is delivered by Hexagon and configured for use with BendingStudio XT software and a TubeInspect cabin.

Compatible TubeInspect systems are:

- TubeInspect 37 XL
- TubeInspect 37 M
- TubeInspect P16.2 HRC
- TubeInspect P8.2 HRC

2.1.1 Overview

The PLC acts as an controller for BendingStudio XT and is connected with the BendingStudio XT PC with a standard ethernet cable and I/O ports are wired to the robot-controller.

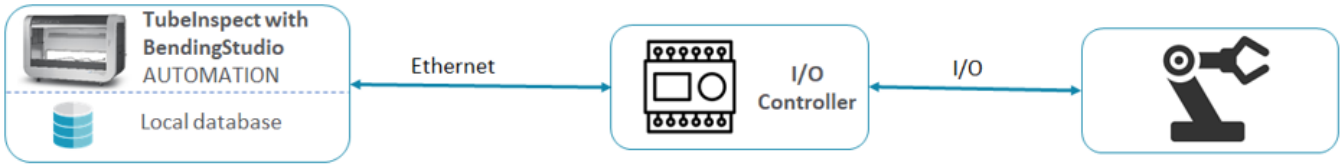


Figure 3: The PLC automation module provides the communication protocol between the BendingStudio XT and the handling system.

The PLC is based on RevPi technology. The standard scope of delivery includes a PLC with Ethernet interface to connect to the TubeInspect PC and a digital I/O module. The firmware is pre-installed; it can be used immediately after the PLC has been integrated into a control cabinet (e.g. in an industrial robot) and after the Ethernet connection has been configured.

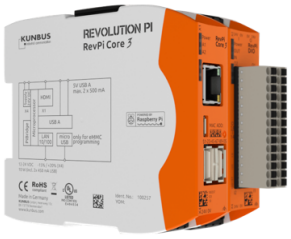


Figure 4: PLC product image

BendingStudio XT commands are triggered with a 24V signal via the digital inputs. The outputs then indicate when and with what status the command execution was completed.

Pin states

Inputs

Pin	Action	Description
Input 1	Calibration	Executes the software-based system calibration. If necessary, the TubeInspect system switches on the side lighting. The system calibration must be executed at the beginning of an automation cycle and at regular intervals.
Input 2	Measurement	Executes a part measurement. The coding pins 7-10 are read out and the measurement for the defined inspection plan of a part is executed. The handling system must move the part to a suitable measurement position.
Input 3	Repeat measurement	Adds another measurement to the current one. To execute a repeat measurement, a first measurement via input 2 must have been triggered before. Coding pins must not change between first and additional measurements. The part can be repositioned before a repeat measurement, e.g. to get a complete measurement.
Input 4	Correction	After a successful measurement, a correction data set is generated for the last measurement result. The output is made by the BendingStudio software in an XML file, see also section Correction result.
Input 5	Measurement report	Creates a measurement report in the defined directory.
Input 6	Export result	Exports an RBS file of the measurement. This file can be processed, for example, by using the BendingStudio XT FileWatcher.
Input 7 -10	Select part	Four pins are available for a coding of parts to be measured. By combining on/off signals, the inspection plan of a part is selected for automatic measurement corresponding to the ID assignment. See also section Assign the ID.

Table 1: Pin assignments of the inputs

Outputs

Pin	Display	Description
Output 1	Status	Shows whether the PLC module accepts input signals. An input pin should only be triggered if the signal *high* is present on this status pin. During the execution of a command, the status is not set: You have to wait with the result query of outputs 5-6 until the pin is set again.
Output 2	Executed command	Indicates whether the last command was executed successfully.
Output 3	Image capture finished	The output pin is set as soon as the images are captured by the TubeInspect. The evaluation of the measurement itself may take longer, but the part can be removed from the TubeInspect from this point onwards. See also section List of broadcasts. The measurement result is available as soon as output pin 1 is set.
Output 4	Measurement completed	Shows a signal when all inspection features have been measured. If the measurement is incomplete, a measurement can be added using the repeat measurement trigger.
Output 5-6	Measurement result	Queries the inspection result. For details, see Control center.

Table 2: Pin assignments of the outputs

2.1.2 Control center

2.1.2.1 Login

The PLC is controlled via a web browser. The address http://plcXXXXXX.local:8080/ opens the dashboard of the PLC control. The ID of the PLC corresponds to the ASN ID of the TubeInspect system, e.g. for ASN240123 the assigned PLC uses the address http://plc240123.local:8080/.

For the configuration, a login is required. The default user name is "admin". On delivery, no password is set.

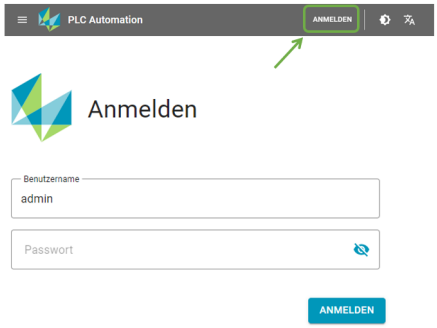


Figure 5: Log in for configuration

After logging in, a password for the admin account can be assigned by clicking the user name. The language of the interface and the colour scheme can also be defined in the header. Clicking the menu list (three horizontal lines) opens the functions. Certain pages are only visible to administrators.

2.1.2.2 Dashboard

The current status information is displayed on the dashboard.

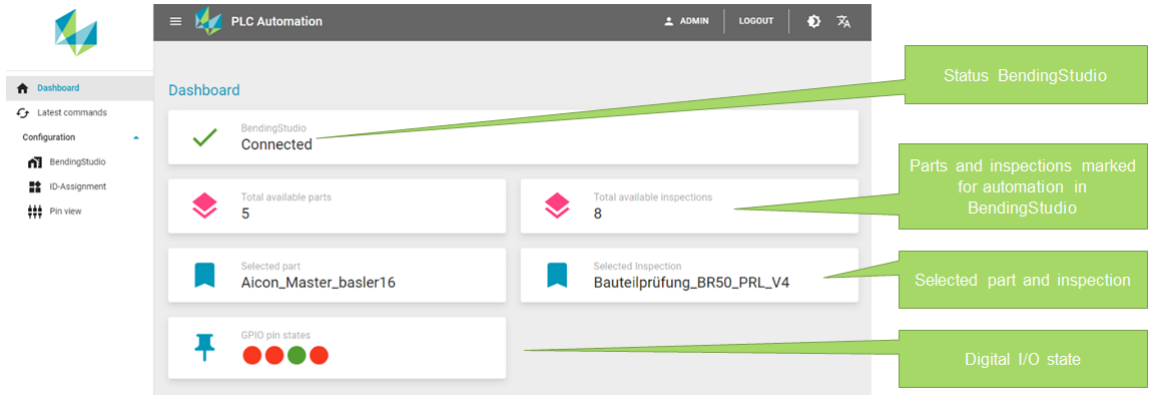


Figure 6: The PLC dashboard

2.1.2.3 Last commands

Lists the last 10 executed commands. This list only is only available at runtime and is not saved permanently.

2.1.2.4 Connect to BendingStudio XT

The connection data to the BendingStudio XT automation server are defined on this page. After saving, a connection attempt is made and the status is displayed.

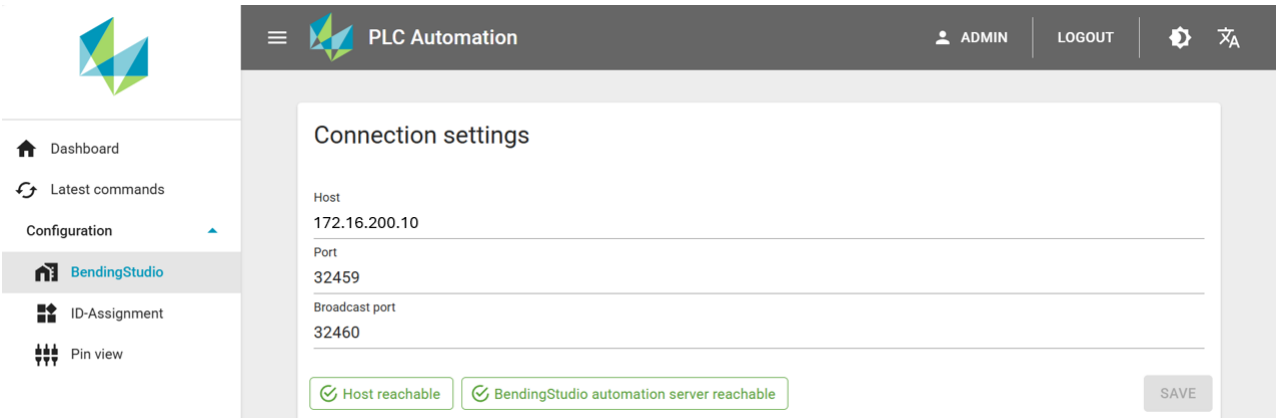


Figure 7: Connection settings

Setting or information	Description
Host	The IPV4 address of the network adapter to which the PLC is connected must be entered in this field.
Port	The port must match the number that was entered in the BendingStudio XT software when configuring the automatic mode.
Broadcast port	The broadcast port must also match the number specified in the BendingStudio XT software when configuring the automatic mode.
Host reachable	Indicates whether the PLC is successfully connected to the specified host.
BendingStudio-Automation server reachable	Indicates whether the BendingStudio XT software is in the automatic mode and whether the server for processing the signals can be reached.

Table 3: Connection messages

INFO Please also observe the hints in the chapter Setup.

2.1.2.5 Assign the ID

On this page, a combination of part and inspection plan is assigned to a digital input pin code.
The available parts for automation are transferred from the BendingStudio XT software to the PLC, they are available in the selection lists on this page. Four input pins are used for coding. This means that a maximum of 16 different parts can be assigned and measured automatically by setting the corresponding input pins. Changes in the assignment must be saved. The assignment is also available after restarting the PLC module. If an assigned part is no longer included in the BendingStudio XT software automation database, a yellow warning symbol is displayed.

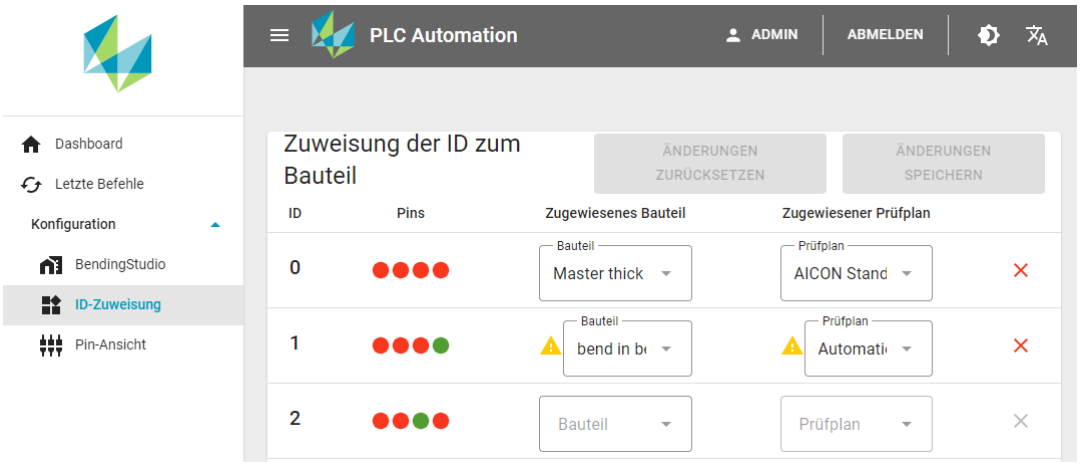


Figure 8: Marking of no longer included parts

2.1.2.6 PIN view

This page shows the current status of the input and output pins. Green means that there is a signal (high).

Pins	Name	Description	
I_1 to I_6	Trigger pins	An action is triggered. After execution, the output pins show the result.	
I_7 to I_10	Coding pins	Binary coding of the ID for selecting a part / inspection plan combination according to the ID assignment table.	
O_1	Status system	1	System is ready. Trigger signals are accepted.
		0	System is busy. Trigger signals are not processed.
O_2	Status triggered command	1	Command execution successful.
		0	Command execution failed.
O_3	Image capture completed	A signal is present as soon as image acquisition in the TubeInspect is complete after a measurement command. The signal indicates that the component can already be removed from the TubeInspect. The system is still busy. The completion of the measurement is indicated by output pin 1, output pins 4-6 provide information about the result of the measurement. **Note:** The pin is only set if the settings for the transmission of broadcasts are defined correctly.	
O_4	Measurement completed	If a part has been completely measured and all defined test criteria have been evaluated, this pin is set to green. In the case of a partial measurement, the pin remains red. In this case, a further measurement via input 2 may be required.	

<table>
<tr><th>Pins</th><th>Name</th><th>Description</th></tr>
<tr><td>O_5 and
O_6</td><td>Status pin for
OK/NOK</td><td>The inspection result is displayed. The following coding applies:
<table>
<tr><th>Inspection result</th><th>O_5 (OK/NOK)</th><th>O_6 (Details)</th></tr>
<tr><td>GREEN
(< intervention limit)</td><td>1</td><td>0</td></tr>
<tr><td>YELLOW
(< tolerance limit)</td><td>1</td><td>1</td></tr>
<tr><td>ORANGE
(< concession limit)</td><td>0</td><td>0</td></tr>
<tr><td>RED
(> concession limit)</td><td>0</td><td>1</td></tr>
</table></td></tr>
</table>

Table 4: Connection messages: Status of input and output pins

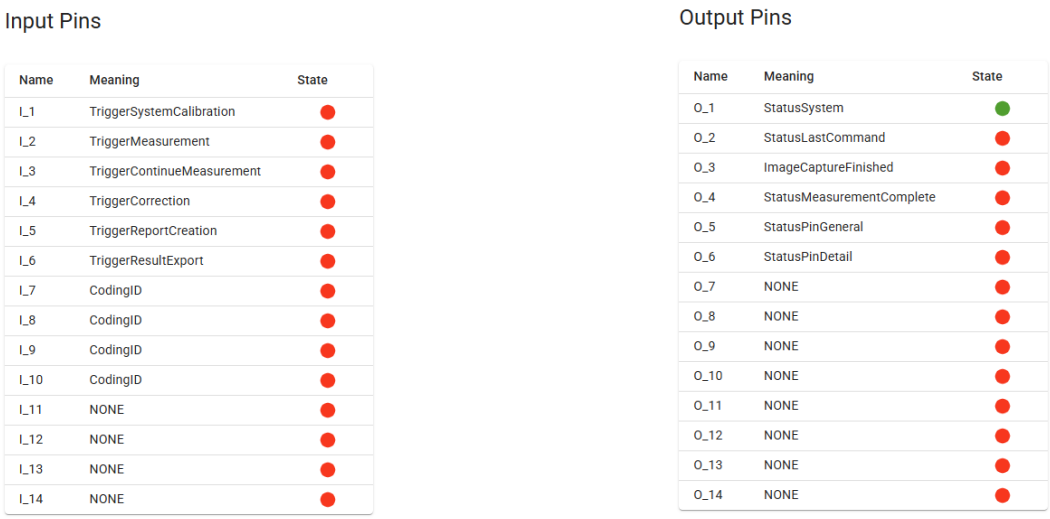


Figure 9: Coding of the inspection results

2.1.3 Setup

2.1.3.1 Configure the network adapter

The PLC module must be directly connected with a network cable to a network adapter of the TubeInspect PC.

INFO The PLC module uses the IP address `172.16.200.240`, subnet mask `255.255.255.0`. The connected network adapter must use an address in the same subnet, e.g. `172.16.200.10`, subnet mask `255.255.255.0`

To configure the network adapter, use Windows short cut `Windows+R` and `type ncpa.cpl`. Select the network adapter and edit TCP/IPv4 properties.

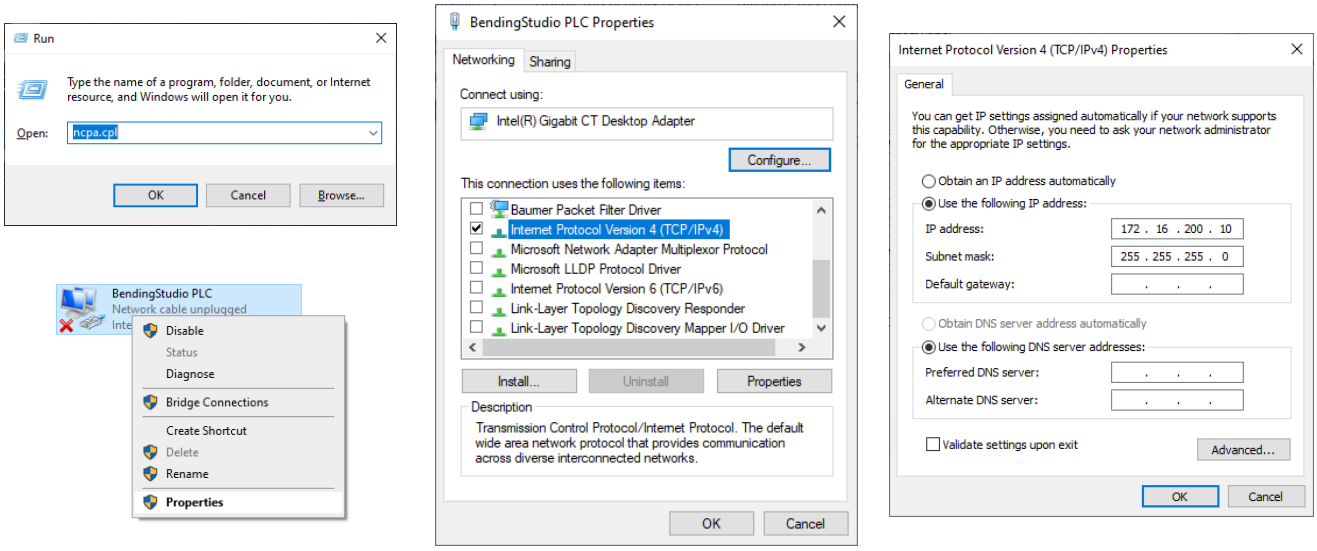


Figure 10: Network adapter settings:

Name of network adapter | Edit TCP/IPv4 Properties | Enter IP-address and subnet mask

INFO **Settings for TubeInspect P8.2 HRC | TubeInspect P16.2 HRC**

PLC modules delivered **before the year 2025** or for the TubeInspect HRC models are using the previous TubeInspect P-series IP address range **`169.254.x.x`** in which the network adapter connected with the PLC module must also be addressed.

Recommendation: Use the IP4 address **`169.254.200.10`** to configure the network adapter for legacy systems.

2.1.3.2 Connection settings

Port settings for communication

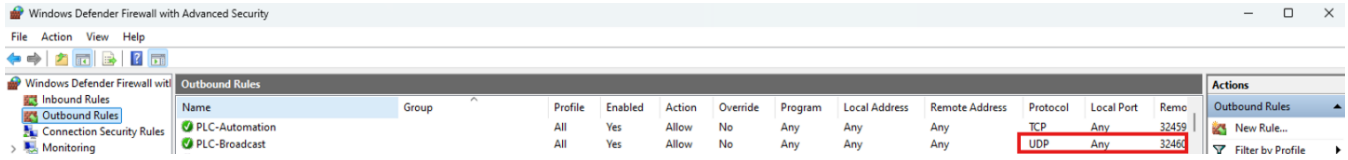
A port must be released to enable the communication with the BendingStudio XT software. By default, port `32459` is used. Changes to the computer's firewall settings may be required. The port must be released for incoming and outgoing rules.

In addition, the port must be activated for a web service. To do this, a `netsh` command must be executed once. The command `EnableAutomationWebServer.cmd` in the installation folder of the BendingStudio XT software (`C:\programme Files\AICON\BendingStudio\Tools`) must be executed with **administrator rights**.

Broadcast port

When using the broadcast signal for image capture, the port for outgoing rules for the UDP protocol must be released in the firewall settings.

In the BendingStudio software, the broadcast port and the broadcast address must also be specified in the automation settings. For the recommended standard address of the network adapter (`172.16.200.10` with subnet mask `255.255.255.0`), this is the address `172.16.200.255`. Online network computers help to determine the correct broadcast address for a specific IP address and subnet mask.



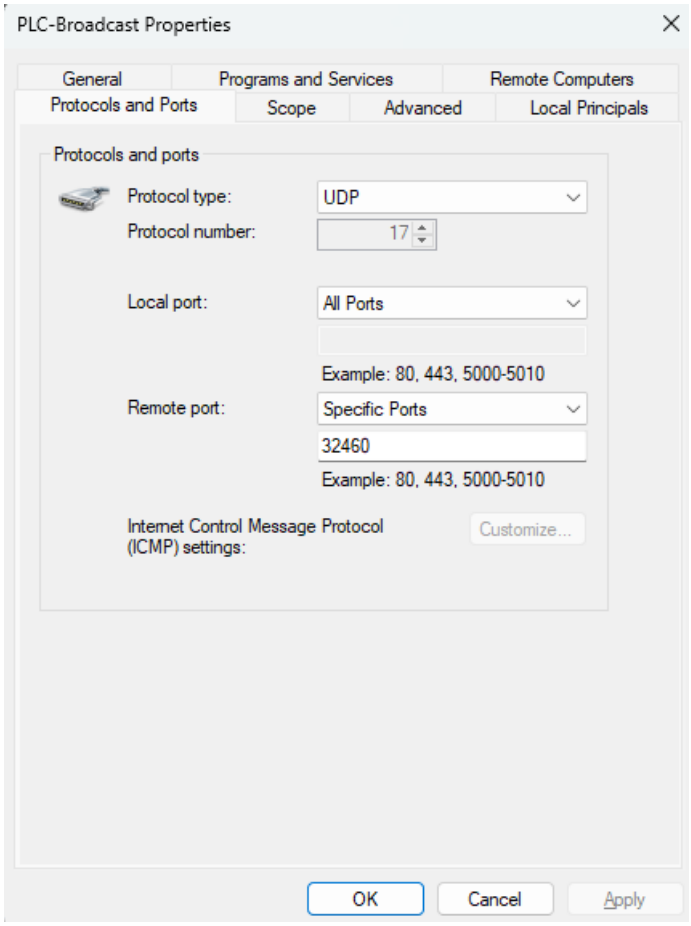


Figure 11: Firewall settings for the broadcast port

Host reachable

In order for the status of the connection display for the host to change to green, ping requests may have to be permitted in the firewall.

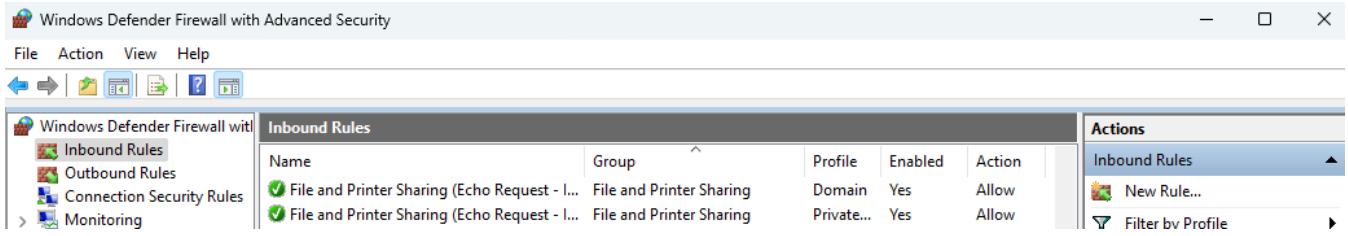


Figure 12: Firewall settings for the accessibility of the host

2.1.4 Example: UR Cobot

The BendingStudio XT PLC module can easily be connected with an UR Cobot. This example describes in keywords how to integrate the PLC into a UR.

1. Integrate the PLC inside the control, e.g. by mounting it on a magnetic holder.
2. Provide 24V external power for the PLC.
3. Connect the external power to the UR control (remove the existing power bridge).
4. Connect the required PLC trigger input signals to the digital outputs of the UR control.
5. Connect the PLC outputs to the digital inputs of the UR control.

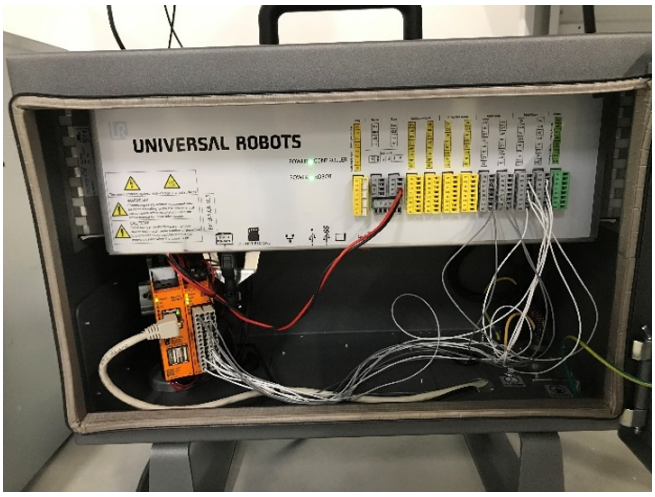


Figure 13: Setup example.
The PLC module is connected via Ethernet to the BendingStudio XT PC. The I/O-connectors of the UR controller are powered with external 24V supply for PLC, the digital inputs and outputs are connected.

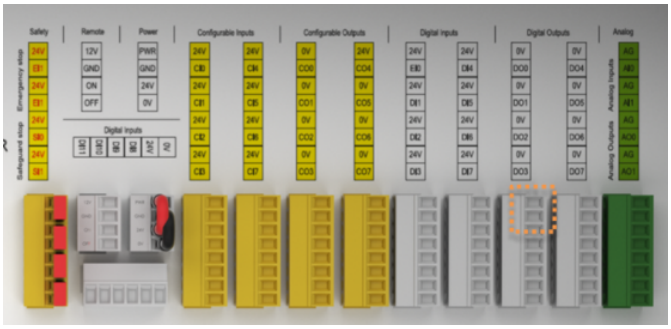


Figure 14: Schematic representation of the connections of a UR control.

Inputs and outputs must be configured in the UR control software. In the programme, trigger signals for UR outputs are set and wait for inputs defined.

INFO A calibration command für the TubeInspect system is required to establish the system's status *Ready for measurement* after starting the automation.
This command should always be placed at the start of a robot programme and be repeated after a defined time period has elapsed, e.g. every hour. Use additional wait commands after setting coding IDs in the PLC to ensure a correct part selection.

2.1.5 PLC box

On request and for an additional charge, the PLC module can be installed in a mini control cabinet with its own 24V power supply for the connection to a handling system. The connection is executed with "Harting" connectors and 5m cable length. Table 5 lists the connector and cable assignment:



Pin	Assignment
1	INPUT 1 - Trigger System Calibration
2	INPUT 2 - Trigger Measurement
3	INPUT 2 - Trigger Continue Measurement
4	INPUT 4 - Trigger Correction
5	INPUT 5 - Trigger Report Creation
6	INPUT 6 - Trigger Result Export
7	INPUT 7 - Coding ID 1
8	INPUT 8 - Coding ID 2
9	INPUT 9 - Coding ID 3
10	INPUT 10 - Coding ID 4
11 - 16	Not used
17	OUTPUT 1 - Status System
18	OUTPUT 2 - Status Last Command
19	OUTPUT 3 - NONE
20	OUTPUT 4 - Status Measurement Complete
21	OUTPUT 5 - Status Result General
22	OUTPUT 6 - Status Result Detail
23	+24 Volt
24	0 Volt

Table 5: Assignment of connectors and cables

2.2 XML interface

The communication is based on a simple XML protocol; protocol messages are transferred over HTTP.

After connecting to the server on port 32459 (default port, another port can be defined in the BendingStudio XT software), the master/controller can send commands to (see Commands) or query data from the server (see Queries). The server acknowledges each command/query with a reply (see Return). Broadcast messages are sent by the server and can be used optionally to receive status information (see Broadcasts).

Prerequisites:

1. A URL registration and a Firewall exception is added for the URL under which the BendingStudio XT automation interface shall run. You can configure these settings with the *Netsh.exe* tool.
   Sample: `netsh http add urlacl url=http://+:32459/ user=Domain\Username`
2. "Except 100-continue" is not supported by the BendingStudio XT automation interface . The master/controller must be configured to send requests in one batch.

NOTICE **Case-sensitive XML parsers**

Most XML parsers are case-sensitive, so be careful when using XML tags.

2.2.1 Queries

2.2.1.1 Coding description

A query is identified by the tag `BSExecQuery` and is defined by the following syntax:

```
<BSExecQuery>
      <QueryID>[PartList, InspectionList]</QueryID>
      <Parameters> // optional
            <Part>[Name or GUID]</Part>
      </Parameters>
</BSExecQuery>
```

The BendingStudio XT automation interface offers a list of queries that can be used to retrieve data. They are identified by a query ID. The following queries are supported:

QueryID	Description
`PartList`	Returns the list of all parts available for automation
`InspectionList`	Returns the list of available inspection plans for a defined part.
`State`	Returns the code `success`, if the BendingStudio server is running.

Table 6: Supported queries

Some queries require a parameter:

QueryID	Parameter	Description
`InspectionList`	Part	The corresponding part as name or GUID.

Table 7: Queries requiring a parameter

Each query is acknowledged with a structure starting with the tag `BSReturn`, see Code samples. The answer contains two identifiers (name and GUID) for every entry. These identifiers can be used to execute other queries or commands.

2.2.1.2 Code samples

1. Lists all parts

```
<BSExecQuery>
    <QueryID>PartList</QueryID>
</BSExecQuery>
```

2. Lists all inspection plans of “MyPart” with name

```
<BSExecQuery>
    <QueryID>InspectionList</QueryID>
    </Parameters>
          <Part>MyPart</Part>
    </Parameters>
</BSExecQuery>
```

3. Lists all inspection plans of “MyPart” with GUID

```
<BSExecQuery>
    <QueryID>InspectionList</QueryID>
    </Parameters>
          <Part>740A47B6-3164-4A64-A850-2C00C83972EE</Part>
    </Parameters>
</BSExecQuery>
```

4. Ping request for optional checks if the server is responding

```
<BSExecQuery>
    <QueryID>State</QueryID>
</BSExecQuery>
```

2.2.2 Commands

2.2.2.1 Coding description

A command is identified by the tag `BSExecCommand` and is defined by the following syntax:

```
<BSExecCommand>
  <CommandID>[Inspect, Correct, Report, Calibrate, ExportRBS]</CommandID>
  <Part>[Name or GUID]</Part>
  <Inspection>[Name or GUID]</Inspection>
  <Result>[Name or GUID]</Result>
  <ResultList>
    <Result>[Name or GUID]</Result>
    ...
    <Result>[Name or GUID]</Result>
  </ResultList>
  <AllResults>true</AllResults>
  <CommandParameters> // optional
    <Comments>xs:string</Comments>
    <ExportPath>xs:string</ExportPath>
  </CommandParameters>
</BSExecCommand>
```

The automation interface offers a list of commands that can be used to trigger actions. They are identified by a command ID. The following commands are supported:

CommandID	Description
`Inspect`	Executes the measurement or continues a measurement. **INFO** In order to continue a measurement, the `Result` attribute must be set.
`Correct`	Executes a correction calculation.
`Report`	Creates a report.
`Calibrate`	Executes a system calibration.
`TestCalibration`	Executes a calibration test.
`ExportRBS`	Exports a result as `*.rbs` file.

Table 8: Supported commands

NOTICE **Overwriting existing reports**

Creating a report overwrites an existing report with the same file name without confirmation.

NOTICE **General recommendations for executing a calibration**

A successful calibration is the basis for every measurement. If a calibration warning or an error occurs (see also Queries, Examples), the system is not ready to execute a measurement!
In general, a calibration command must be executed in the following situations:

1. Always when the automation is started

   and

2. On a regular basis in order to guarantee a frequent calibration of the system.

   Recommendation:

   For automation, select an interval of 1 or 2 hours.
   This means that after the interval is elapsed, the automation interface will return the error code 313. It is required to send a calibration command via the interface (see Queries, Examples).

   The interval is defined in the device properties of the TubeInspect systems (see INFO below).

3. Always, if a warning or an error occurs (see Queries, Examples).
   Issues which might end up in a warning or error could be, for example:
   - Camera failed or is defect.
   - Camera has moved due to vibrations.
   - If too many reference targets are covered, e.g. by the robot arm and/or the gripper.

NOTICE **General recommendations for executing a calibration test**

In the automation mode, the calibration test is disabled by default. The calibration test can be executed by sending the command `TestCalibration`. The calibration test checks if there have been any movements in measurement cameras

by shocks or vibrations by checking the position of the reference targets. A handling system like a robot often hides targets and the automatic test before each measurement makes no sense.
If the TubeInspect system is used in a rough environment a frequent use of the calibration test is recommended. In comparison to a calibration, the test needs no side lighting turned on and saves time.

INFO The calibration interval is defined in BendingStudio in the Hardware administration app.

A. Log in as user “hardware”.
B. Open the BendingStudio *Hardware administration*.
C. Open *Device properties* and go to the *Calibration* tab.
D. Select one of the available options for the parameter *Calibration interval*.

Some commands require additional attributes:

Attribute	Description
`Part`	The relevant part as name or GUID.
`Inspection`	The relevant inspection as name or GUID.
`Result`	The relevant result as name or GUID.
`ResultList`	A list of relevant results as name or GUID.
`AllResults`	A flag that can be set to TRUE.

Table 9: Commands requiring additional attributes

For some commands optional parameters can be added:

Parameter	Description
`Comments`	The value is set as comment in the result.
`ExportPath`	It can be set as directory path or as file path. It overwrites the settings of the configuration.

Table 10: Optional parameters for command

CommandID	Additional attributes						
	Part	*Inspection*	*Result*	*ResultList*	*AllResults*	*Comments (optional)*	*ExportPath (optional)*
`Inspect`	x	x	x			x	
`Correct`	x		x	x	x		
`Report`	x		x				x
`ExportRBS`	x		x				

Table 11: Additional attributes for commands

Each command is acknowledged with a structure starting with the tag `BSReturn`, see Return. The answer contains two identifiers (name and GUID) for every entry. These identifiers can be used to execute other queries or commands.

2.2.2.2 Code samples

1. Start an inspection

```
<BSExecCommand>
   <CommandID>Inspect</CommandID>
   <Part>MyPart</Part>
   <Inspection>MyInspection</Inspection>
   <CommandParameters>
      <Comments>More information</Comments>
   </CommandParameters>
</BSExecCommand>
```

2. Continue an inspection

```
<BSExecCommand>
   <CommandID>Inspect</CommandID>
   <Part>MyPart</Part>
   <Inspection>MyInspection</Inspection>
   <Result>MyResult</Result>
   <CommandParameters>
      <Comments>More information</Comments>
   </CommandParameters>
</BSExecCommand>
```

3. Compute correction values

a) Based on a single result

```
<BSExecCommand>
   <CommandID>Correct</CommandID>
   <Part>MyPart</Part>
   <Result>MyResult</Result>
</BSExecCommand>
```

b) Based on multiple specific results

```
<BSExecCommand>
   <CommandID>Correct</CommandID>
   <Part>MyPart</Part>
   <ResultList>
       <Result>MyResult1</Result>
       <Result>MyResult2</Result>
       ...
   </ResultList>
</BSExecCommand>
```

c) Based on all existing results

```
<BSExecCommand>
   <CommandID>Correct</CommandID>
   <Part>MyPart</Part>
   <AllResults>true</AllResults>
</BSExecCommand>
```

4. Create a report

```
<BSExecCommand>
   <CommandID>Report</CommandID>
   <Part>740A47B6-3164-4A64-A850-2C00C83972EE</Part>
   <Result>MyResult</Result>
   <CommandParameters>
      <ExportPath>C:\Temp\%Y_%m_%d_%H_%M_%S_Result_%r.xps</ExportPath>
   </CommandParameters>
</BSExecCommand>
```

5. Execute a system calibration

```
<BSExecCommand>
   <CommandID>Calibrate</CommandID>
</BSExecCommand>
```

6. Execute a system calibration test

```
<BSExecCommand>
   <CommandID>TestCalibratio</CommandID>
</BSExecCommand>
```

7. Execute an *.rbs export of a result

```
<BSExecCommand>
   <CommandID>ExportRBS</CommandID>
   <Part>740A47B6-3164-4A64-A850-2C00C83972EE</Part>
   <Result>MyResult</Result>
</BSExecCommand>
```

2.2.3 Return

2.2.3.1 Coding description

The BendingStudio XT automation interface acknowledges each query or command. An answer is identified by the tag `BSReturn` and is defined by the following syntax:

```
<BSReturn>
     <Header>
       Content of original request
     </Header>
     <ReturnCode>[Success, Failed]</ReturnCode>
     <Payload>
       <Object>
          ...
       </Object>
       OR
       <ObjectList>
         <ObjectType>[Part, Inspection, Result, Correction]</ObjectType>
         <Object>
            ...
         </Object>
         ...
       </ObjectList>
     </Payload>
</BSReturn>
```

The answer corresponding to a request contains the original request and a return code. Additional information depends on the request.

Information	Description
`Header`	Contains the original request.
`ReturnCode`	Contains the return code (see ReturnCode, ErrorCode and WarningCode).
`Payload`	Contains the requested information.
`ErrorCode`	Contains an error code if `ReturnCode` is set to *Failed* (see Code samples).

Table 12: Additional information

The payload contains individual information for each type. Requested data types are *Part, Inspection, Result, Correction* and *Report*. It may contain a single object (`Object`) or a list of objects (`ObjectList`).

Payload	
Attribute	**Description**
`Object`	Contains the data generated for the request (see Payload - Object).
`ObjectList`	Contains the list of the objects. The type of containing objects is set in `ObjectType`.

Table 13: Requested data types for payload

2.2.3.2 ReturnCode, ErrorCode and WarningCode

Each answer returns a return code for the processing of the request.

ReturnCode	
Value	**Description**
`success`	The request was processed successfully.
`success with warning`	The request was processed successfully, but a warning is generated.
`failed`	The request failed.

Table 14: Return codes

INFO **Recommendation**

If an error or warning code occurs, please check the syntax and the send information in general. Afterwards, see the recommended actions in the following tables.

In case of a failure, an error code is returned. The error codes are divided into system errors (100), structural errors (200), semantic errors (300) and XML semantic errors (400).

ErrorCode		
Value	**Description**	**Recommended action**
100	A database request could not be executed.	Fatal error: The automation must be stopped. Operator interaction required.
101	The server no longer responds.	
102	The server request was aborted.	
200	The specified query is invalid.	Verify syntax or information of the transferred structures.
201	The specified command is invalid.	
202	Not all required parameters are set for the request.	
300	The measurement failed.	Put the part into a separate bin for a manual check or try to measure it again.
301	The measurement no longer responds.	
302	The measurement cannot be started.	
303	The correction failed.	Correction could not be calculated. User interaction required.
304	The report generation failed.	Storage path is not available. Please check its availability.
305	The part does not exist.	Verify syntax or information of the transferred structures (see Queries).
306	The inspection plan does not exist.	

ErrorCode		
Value	**Description**	**Recommended action**
307	The result does not exist.	
308	The correction does not exist.	
309	The continued measurement failed.	Put the part into a separate bin for a manual check or try to measure it again.
310	The calibration has failed.	Fatal error: System does not run properly. A calibration is required.
311	The calibration check has failed.	
312	The measurement system operates in emergency mode.	
313	A calibration is required.	
314	The file could not be saved.	Storage path is not available. Please check its availability.
400	XML: The request is invalid.	Verify syntax or information of the transferred structures.
401	XML: No valid Exec element exists.	
402	XML: No valid ID element exists.	
403	XML: The element does not contain any entries.	
404	XML: The parameter element does not contain any entries.	

Table 15:Error codes

WarningCode		
Value	**Description**	**Recommended action**
1000	The calibration check is faulty.	Perform a calibration. If the warning occurs again, the automation must be stopped. Operator interaction required.
1001	One or more cameras are defect.	
1002	The maintenance is due soon.	Contact your local support.
1003	The maintenance date is reached.	
1004	Switching of the side light failed.	Fatal error. The automation must be stopped. Operator interaction required.
1005	CAD adapter measurement failed.	Put the part into a separate bin for a manual check or try to measure it again.
1006	The measurement system operates in emergency mode. Calibration failed for one or more cameras.	Check the system and replace defect cameras.

Table 16: Warning codes

2.2.3.3 Payload - Object

An object is described by a type and identifiers – name and/or GUID and/or path. The GUID and name can be used to parameterise queries and commands.

The following data objects are defined as payload of query or command execution:

```
<Object>
     <Type>[Part, Inspection, Result, Correction, Report]</Type>
     <Name>xs:string</Name>
     <GUID>xs:string</GUID>
     <Path>xs:string</Path>
</Object>
```

Attribute	Description
`Type`	Defines the type of the object. Possible values are: `Part, Inspection, Result, Correction, Report.`
`Name`	Contains the name of the object.
`GUID`	Contains the GUID of the object (GUID: Unique ID to identify the object).
`Path`	Contains the path of the object in the automation database.
`Table`	Optionally, the content depends on the `Type` attribute. The content of the `Table` attribute is structured like a normal table. It is divided into rows and columns. Each row has an identifier (ID), either numbered in ascending order or with a defined text. The columns contain the values.

Table 17: Attributes

INFO Optionally, some objects contain additional information. This depends on the object type (`Result, Correction, Report`).

2.2.3.3.1 Measurement result

The measurement result payload is defined by the following syntax:

```
<Object>
 <Type>Result</Type>
 <Name> xs:string </Name>
 <GUID> xs:string </GUID>
 <Path> xs:string </Path>
 <ResultValue> xs:string </ResultValue>
 <MaxTolerance> xs:string </MaxTolerance>
 <IsCompleted> yes/no </IsCompleted>
 <Table>
       <Name>Result</Name>
       <Data>
             <Row id="Component">
                   <Column id="CompName"> xs:string </Column>
                   <Column id="Length"> xs:string </Column>
                   <Column id="LengthDev"> xs:string </Column>
                   <Column id="EndToEnd"> xs:string </Column>
                   <Column id="EndToEndDev"> xs:string </Column>
             </Row>
             <Row id="SheathCriteria">
                   <Column id="CritName"> xs:string </Column>
                   <Column id="CompName"> xs:string </Column>
                   <Column id="StartType"> xs:string </Column>
                   <Column id="StartValue"> xs:string </Column>
                   <Column id="EndType"> xs:string </Column>
                   <Column id="EndValue"> xs:string </Column>
             </Row>
       </Data>
 </Table>
</Object>
```

If an inspection criteria of type *Sheath deviation* with a *cut length* for end processing is defined, the returned result includes an additional block in the xml-message with cut length information:

```
<Object>
 ...
 <Table>
       ...
             <Row id="SheathCriteria">
                   <Column id="CritName"> xs:string </Column>
                   <Column id="CompName"> xs:string </Column>
                   <Column id="StartType"> xs:string </Column>
                   <Column id="StartValue"> xs:string </Column>
                   <Column id="EndType"> xs:string </Column>
                   <Column id="EndValue"> xs:string </Column>
             </Row>
       ...
 </Table>
</Object>
```

Attribute	Description	
`ResultValue`	Returns the measurement result, range [0.0, 4.0]. For a detailed description of the result value, see Result evaluation.	
`MaxTolerance`	Returns the maximum tolerance utilisation of the result in percent.	
`IsCompleted`	Returns whether all components were measured completely (yes/no).	
`Table`	Returns a data for each component and sheath criteria.	
	RowType – Component	
	CompName	Name of the component.
	Length	Rolled out length of the measured component. Unit [mm]
	LengthDev	Deviation to the nominal rolled-out length. Unit [mm]
	EndToEnd	End-to-end distance of the measured component. Unit [mm]
	EndToEndDev	Deviation to the nominal end-to-end distance. Unit [mm]
	RowType – SheathCriteria	
	CritName	Name of the sheath criteria.
	CompName	Name of the component.
	StartType	Type of start: `CutLength` or `CutInGage`
	StartValue	Value of cut at start. Unit [mm]
	EndType	Type of end: `CutLength` or `CutInGage`
	EndValue	Value of cut at end. Unit [mm]

Table 18: Attributes (Measurement result)

2.2.3.3.2 Correction result

The correction result is defined by the following syntax:

```
<BSReturn>
  <Header>
    <BSExecCommand>
      <CommandID>Correct</CommandID>
      <Part>B953A786-9691-40F6-9409-99E81515E930</Part>
      <Result>2019/09/02-11:29:42</Result>
    </BSExecCommand>
  </Header>
  <ReturnCode>success</ReturnCode>
  <Payload>
    <Object>
      <Type>Correction</Type>
      <Name>Corr_2019/09/02-11:41:41</Name>
      <GUID>0E812235-7249-434D-89C0-65A42CEE657F</GUID>
      <Path>/parts/B953A786-9691-40F6-9409-99E81515E930/corrections/0E812235-7249-
434D-89C0-65A42CEE657F.obj</Path>
      <Table>
        <Name>Correction</Name>
        <Data>
            <Row id="0">
                  <Column id="BE">1</Column>
                  <Column id="Push">37.22</Column>
                  <Column id="PushDev">1.05</Column>
                  <Column id="Bend">1.65</Column>
                  <Column id="BendDev">0.09</Column>
                  <Column id="Rotate">0.43</Column>
                  <Column id="RotateDev">-0.12</Column>
                  <Column id="Radius">25.62</Column>
                  <Column id="RadiusDev">-0.05</Column>
            <Row>
                  ...
            </Row>
            ...
        </Data>
      </Table>
    </Object>
  </Payload>
</BSReturn>
```

Attribute	Description	
`Table`	Returns a list of rows and columns containing the correction values. The ID in a row describes the type of the value (see Examples).	
	`BE`	Number of bending element.

Attribute	Description	
	`Push`	Value of push. Unit [mm]
	`PushDev`	Deviation to nominal push. Unit [mm]
	`Bend`	Value of bending angle. Unit [rad]
	`BendDev`	Deviation to nominal bending angle. Unit [rad]
	`Rotate`	Value of rotation angle. Unit [rad]
	`RotateDev`	Deviation to nominal rotation angle. Unit [rad]
	`Radius`	Value of radius. Unit [mm]
	`RadiusDev`	Deviation to nominal radius. Unit [mm]

Table 19: Attributes (Correction result)

2.2.3.3.3 Report

The report is defined by the following syntax:

```
<Object>
        ...
        <Language> xs:string </Language>
</Object>
```

Attribute	Description
`Language`	Returns the language of the report object as region code (English – en, etc.).

Table 20: Attributes (Report)

2.2.3.4 Examples

2.2.3.4.1 Examples for answering queries

The following list shows examples for answers to different queries:

1. Return of query `State`

```
<BSReturn>
  <Header>
    <BSExecQuery>
      <QueryID>State</QueryID>
    </BSExecQuery>
  </Header>
  <ReturnCode>success</ReturnCode>
</BSReturn>
```

2. Return of query `PartList`

```
<BSReturn>
  <Header>
     <BSExecQuery>
        <QueryID>PartList</QueryID>
     </BSExecQuery>
  </Header>
  <ReturnCode>success</ReturnCode>
  <Payload>
     <ObjectList>
        <ObjectType>Part</ObjectType>
        <Object>
           <Type>Part</Type>
           <Name>MyPart1</Name>
           <GUID>B953A786-9691-40F6-9409-99E81515E930</GUID>
           <Path>/parts/B953A786-9691-40F6-9409-99E81515E930/part.obj</Path>
        </Object>
        <Object>
           <Type>Part</Type>
           <Name>SecondPart</Name>
           <GUID>F12ACB2A-91FE-4374-9B26-96611E40A53B</GUID>
           <Path>/parts/F12ACB2A-91FE-4374-9B26-96611E40A53B/part.obj </Path>
        </Object>
     </ObjectList>
  </Payload>
</BSReturn>
```

3. Return of query `InspectionList`

```
<BSReturn>
  <Header>
    <BSExecQuery>
      <QueryID>InspectList</QueryID>
      <Parameters>
        <Part>MyPart</Part>
      </Parameters>
  </BSExecQuery>
  </Header>
  <ReturnCode>success</ReturnCode>
  <Payload>
    <ObjectList>
      <ObjectType>Inspection</ObjectType>
      <Object>
        <Type>Inspection</Type>
        <Name>GaugeInspection</Name>
        <GUID>43C77B28-DC10-4268-B82E-54D0B6B0819C</GUID>
        <Path>/parts/B953A786-9691-40F6-9409-99E81515E930/inspections/43C77B28-DC10-
4268-B82E-54D0B6B0819C.obj</Path>
      </Object>
      <Object>
        <Type>Inspection</Type>
        <Name>TestInspection</Name>
        <GUID>9A465E07-F50C-458D-B1C5-DD6C86534377</GUID>
        <Path>/parts/B953A786-9691-40F6-9409-99E81515E930/
inspections/9A465E07-F50C-458D-B1C5-DD6C86534377.obj</Path>
      </Object>
    </ObjectList>
  </Payload>
</BSReturn>
```

2.2.3.4.2 Example for answering commands

The following list shows examples for answers to different commands:

1. Return of command `Inspect`

```
<BSReturn>
  <Header>
    <BSExecCommand>
      <CommandID>Inspect</CommandID>
      <Part>B953A786-9691-40F6-9409-99E81515E930</Part>
      <Inspection>43C77B28-DC10-4268-B82E-54D0B6B0819C</Inspection>
    </BSExecCommand>
  </Header>
  <ReturnCode>success</ReturnCode>
  <Payload>
    <Object>
      <Type>Result</Type>
      <Name>2019/09/02-11:29:42</Name>
      <GUID>1E8D0F7E-8DA2-4295-B3F4-5B87B4BDFC16</GUID>
      <Path>/parts/B953A786-9691-40F6-9409-99E81515E930/results/1E8D0F7E-8DA2-4295-
        B3F4-5B87B4BDFC16.obj</Path>
      <ResultValue>4.00</ResultValue>
      <MaxTolerance>1356.55</MaxTolerance>
      <IsCompleted>yes</IsCompleted>
      <Table>
        <Name>Result</Name>
        <Data>
          <Row id="Component">
              <Column id="CompName">PBRRComponent</Column>
              <Column id="Length">1339.39</Column>
              <Column id="LengthDev">18.62</Column>
              <Column id="EndToEnd">511.23</Column>
              <Column id="EndToEndDev">-5.40</Column>
          </Row>
          <Row id="SheathCriteria">
              <Column id="CritName">Sheath deviation</Column>
              <Column id="CompName">PBRRComponent</Column>
              <Column id="StartType">CutInGage</Column>
              <Column id="StartValue">7.31</Column>
              <Column id="EndType">CutLength</Column>
              <Column id="EndValue">-21.60</Column>
          </Row>
        </Data>
      </Table>
    </Object>
  </Payload>
</BSReturn>
```

2. Return of command `Correct`

```
<BSReturn>
  <Header>
    <BSExecCommand>
      <CommandID>Correct</CommandID>
      <Part>B953A786-9691-40F6-9409-99E81515E930</Part>
      <Result>2019/09/02-11:29:42</Result>
    </BSExecCommand>
  </Header>
  <ReturnCode>success</ReturnCode>
  <Payload>
    <Object>
      <Type>Correction</Type>
      <Name>Corr_2019/09/02-11:41:41</Name>
      <GUID>0E812235-7249-434D-89C0-65A42CEE657F</GUID>
      <Path>/parts/B953A786-9691-40F6-9409-99E81515E930/corrections/0E812235-7249-
          434D-89C0-65A42CEE657F.obj</Path>
      <Table>
        <Name>Correction</Name>
        <Data>
          <Row id="0">
              <Column id="BE">1</Column>
              <Column id="Push">37.22</Column>
              <Column id="PushDev">1.05</Column>
              <Column id="Bend">1.65</Column>
              <Column id="BendDev">0.09</Column>
              <Column id="Rotate">0.43</Column>
              <Column id="RotateDev">-0.12</Column>
              <Column id="Radius">25.62</Column>
              <Column id="RadiusDev">-0.05</Column>
          </Row>
          <Row id="1">
              <Column id="BE">2</Column>
              <Column id="Push">47.70</Column>
              <Column id="PushDev">0.32</Column>
              <Column id="Bend">1.30</Column>
              <Column id="BendDev">0.19</Column>
              <Column id="Rotate">3.14</Column>
              <Column id="RotateDev">-0.01</Column>
              <Column id="Radius">26.98</Column>
              <Column id="RadiusDev">0.66</Column>
          </Row>
          <Row>
              ...
          </Row>
          ...
        </Data>
      </Table>
    </Object>
  </Payload>
</BSReturn>
```

3. Return of command `Report`

```
<BSReturn>
  <Header>
    <BSExecCommand>
      <CommandID>Report</CommandID>
      <Part>B953A786-9691-40F6-9409-99E81515E930</Part>
      <Result>2019/09/02-11:29:42</Result>
    </BSExecCommand>
  </Header>
  <ReturnCode>success</ReturnCode>
  <Payload>
    <ObjectList>
      <ObjectType>Report</ObjectType>
      <Object>
        <Type>Report</Type>
        <Path>C:\Temp\MyPart-admin-20190902-112942_en.xps</Path>
        <Language>en</Language>
      </Object>
      <Object>
        <Type>Report</Type>
        <Path>C:\Temp\MyPart-admin-20190902-112942_es.xps</Path>
        <Language>es</Language>
      </Object>
    </ObjectList>
  </Payload>
</BSReturn>
```

4. Return of command `ExportRBS`

```
<BSReturn>
  <Header>
    <BSExecCommand>
      <CommandID>ExportRBS</CommandID>
      <Part>B953A786-9691-40F6-9409-99E81515E930</Part>
      <Result>2019/09/02-11:29:42</Result>
    </BSExecCommand>
  </Header>
  <ReturnCode>success</ReturnCode>
</BSReturn>
```

The following example shows the return of an invalid command:

1. Return of command `Inspect` with invalid part name

```
<BSReturn>
   <Header>
      <BSExecCommand>
         <CommandID>Inspect</CommandID>
         <Part>NotMyPart</Part>
         <Inspection>43C77B28-DC10-4268-B82E-54D0B6B0819C</Inspection>
      </BSExecCommand>
   </Header>
   <ReturnCode>failed</ReturnCode>
   <ErrorCode>305</ErrorCode>
</BSReturn>
```

2.2.3.5 Broadcasts

In the automation mode, the BendingStudio server sends broadcast messages via the TCP/IP interface.

For transferring status information, the client must listen to broadcast messages transmitted via a separate port. The port number is configured within the configuration in the automation app.

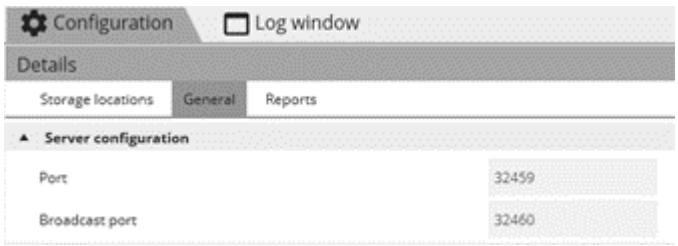


Figure 15 :Port number

If the socket connection is open, the client can listen on this port to messages described in List of broadcasts.

INFO Broadcasts are also listed in the Test client tool. The tool can be run in parallel to learn about when broadcasts are sent by the automation interface.

2.2.3.5.1 List of broadcasts

1. Image acquisition broadcast

This broadcast is sent after an inspection command as soon as the image acquisition from the TubeInspect P-series is completed. After the broadcast has been received, the part can be removed from the TubeInspect system, even if the inspection command has not been completed yet. The time needed from BendingStudio XT software for image measurement and result evaluation can be used as handling time for unloading and loading.

Broadcast ID: MeasurementStatus

Payload:

```
<BSBroadcast>
   <BroadcastID>MeasurementStatus</BroadcastID>
   <Payload>
      <MeasurementStatus>QueryImagesFinished</MeasurementStatus>
      <PartGUID>AB9F26EC-FE7B-4C7D-A5A7-0244B5C1BA3D</PartGUID>
      <InspectionGUID>749A00BE-6A1E-4094-8DAA
                      9301508C3D36</InspectionGUID>
   </Payload>
</BSBroadcast>
```

2. Server status broadcast

This broadcast is sent if the BendingStudio automation server is started or shut down.

```
<BSBroadcast>
   <BroadcastID> ServerStatus</BroadcastID>
   <Payload>
      <ServerStatus>ServerAvailable</ServerStatus>
   </Payload>
</BSBroadcast>


<BSBroadcast>
   <BroadcastID> ServerStatus</BroadcastID>
   <Payload>
      <ServerStatus>ServerShutdown</ServerStatus>
   </Payload>
</BSBroadcast>
```

2.3 Automation Tester

With the tool *BendingStudio Automation Tester*, XML messages can be sent to the BendingStudio server and returns are displayed. It is useful to test an XML code, to create queries and commands or to learn the syntax of returns from the automation interface.

2.3.1 How to use the tester

The *BendingStudio Automation Tester* is included in the installer of the automation interface documentation.

The default installation folder is
`<Documents>\BendingStudio_Automation_Demo\HttpClient`.

The test client comes with a user interface. After the initialisation, the following screen contains test functions described below.

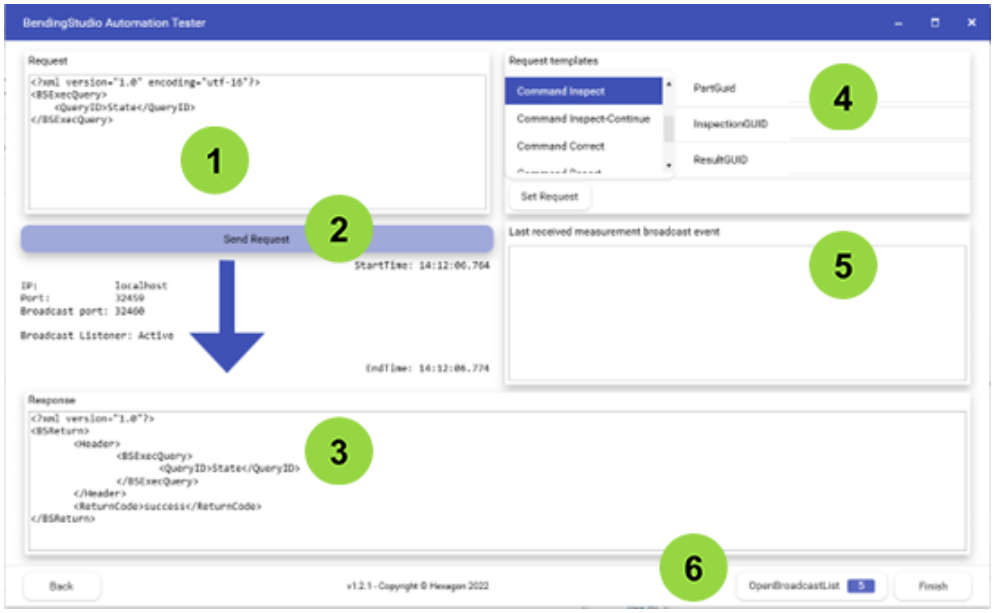


Figure 16: Functions of the BendingStudio Automation Tester

Table 21 describes the functions of the Automation Tester:

No. in Figure 16	Description
1	Text field for request. XML code requests can be typed or pasted. Requests are also created by the Automation Tester, see also 4.
2	Button to send a request.
3	Text field for responses from BendingStudio server or error messages.
4	Templates for requests. Select a template, optionally add payloads and create the XML code by clicking *Set*.
5	Listing of inspection broadcasts.
6	Function bar with • return to port definition, • open a complete broadcast list or • exit the Automation Tester.

Table 21: Functions of the BendingStudio Automation Tester

2.4 Tips and tricks for the start

2.4.1 Result evaluation

BendingStudio XT provides measurement results from different inspection criteria, e.g. sheath deviation or functional dimensions as distances or angles.

The results are returned to the controller summarised as a result value. The result value quickly shows if the measured part is inside tolerance. The value ranges between 0 and 4, **the part is inside the tolerance with value ≤ 2**. Additionally, BendingStudio XT calculates the maximum tolerance utilisation. This is a percentage related to the tolerance limit. For example, a percentage of 95% means the part is inside the tolerance, but close to the limit (100%). The result value also shows the result classification (image and table below).

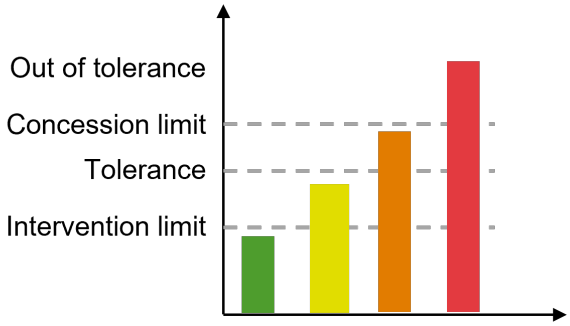


Figure 17 :Result values

To check in the automation process if the part has been completely measured, the `IsCompleted` attribute must be checked as well to ensure all inspection criteria are evaluated. Therefore, a result evaluation to decide whether a part is sorted into an OK or NOK container should include:

- The Attribute `IsCompleted` to get information on the completeness of measurement. Depending on the result, the controller can decide if the part should be measured again or sorted to a special container.
- The Attribute `ResultValue` to get information on the result classification.
- The attribute `MaxTolerance` is an alternative for the attribute `ResultValue`.

The tables below show details about result classification and return values from the automation interface.

If the attribute `IsCompleted` returns `YES`, the complete tube has been measured with all attachments. The evaluation of the returned `ResultValue` and `MaxTolerance` is based on the measurement results of all criteria for the whole part:

Symbol	Tolerance level	Returns of ReturnValue	Returns of MaxTolerance [%]
	Within the tolerance range. Part is OK.	[0;1]	[0;intervention limit]
	Within the intervention limit. Part is OK, but there are deviations above the intervention limit.	]1;2]	]intervention limit; 100]
	Above the concession limit. Part is not OK - there are deviations above the tolerance, but within the concession limit.	]2;3]	]100; concession limit]
	Outside the tolerance range. Part is not OK.	]3;4]	]concession limit; ∞]

Table 22: Attribute `IsCompleted`: `Yes`

If the attribute `IsCompleted` returns `NO`, only a section of the tube is measured. The evaluation of the returned `ResultValue` and `MaxTolerance` is based only on criteria for the measured section:

Symbol	Tolerance level	Return of ResultValue	Return of MaxTolerance [%]
	Within the tolerance range. Section of part is OK.	[0;1]	[0;intervention limit]
	Within the intervention limit. Section of part is OK, but there are deviations above the intervention limit.	]1;2]	]intervention limit; 100]

Symbol	Tolerance level	Return of ResultValue	Return of MaxTolerance [%]
	Above the concession limit. Section of part is not OK - there are deviations above the tolerance, but within the concession limit.	]2;3]	]100;concession limit]
	Outside the tolerance range. Section of part is not OK.	]3;4]	]concession limit; ∞]

Table 23: Attribute `IsCompleted`: `No`

2.4.2 Automatic log-in

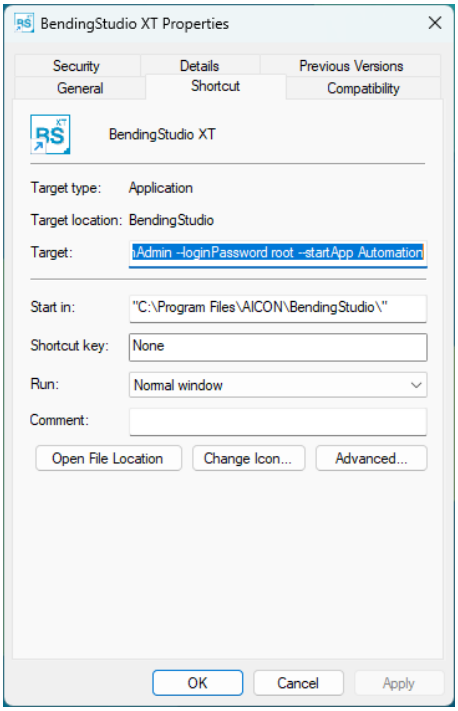
When TubeInspect is used in automation mode, an automatic login in BendingStudio XT is helpful, so that the software can be started from the master controller.

Therefore, the BendingStudio XT executable offers the following command line options:

Option	Description
`--help, -h, /h, -? /?`	Display help text. Example: `Appstudio.exe --help`
`--loginName`	Defines the user name for login. If only this option is used, the user still needs to type his password to login. Example: `Appstudio.exe --loginName SystemAdmin`
`--loginPassword`	Defines the password of the user for login. If both, login name and login password are defined the specified user is logged in after starting BendingStudio XT. Example: `Appstudio.exe --loginName SystemAdmin --loginPassword root`

Option	Description
`--startApp`	Option to start an App after login. Supported arguments are `Automation` and `InspectionAdmin`. Example: `Appstudio.exe --loginName SystemAdmin --loginPassword root --startApp Automation`
`--startAppArg`	Option to start the automation application. The option is only used if the argument `--startApp` is set to `Automation`. Supported arguments are `StartServer` and `StartServerAndLock` to start the automation server or to start the automation server and lock the screen. Example: `Appstudio.exe --loginName SystemAdmin --loginPassword root --startApp Automation --startAppArg StartServer`

The start-up option can be added to the BendingStudio start link on the desktop. Right-click the link, select *Properties* and add the arguments to the *Target*.



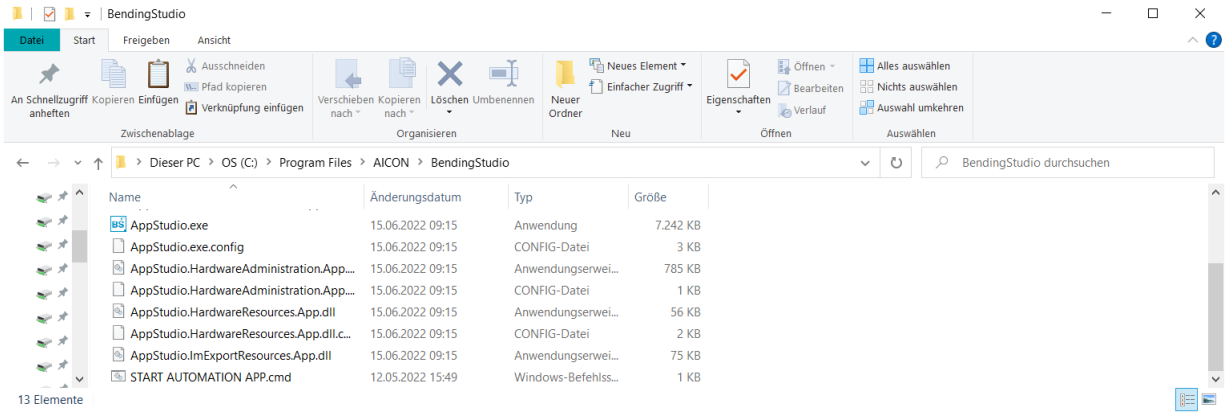
Another option is to configure a script which includes a command to start BendingStudio with the specific options. The script can be executed by the master controller of the automation or added to the Windows task scheduler:

1. Configure a batch script for starting the BendingStudio XT App.
   Name the script file e.g. `STARTBS.cmd`

   Example for the script:

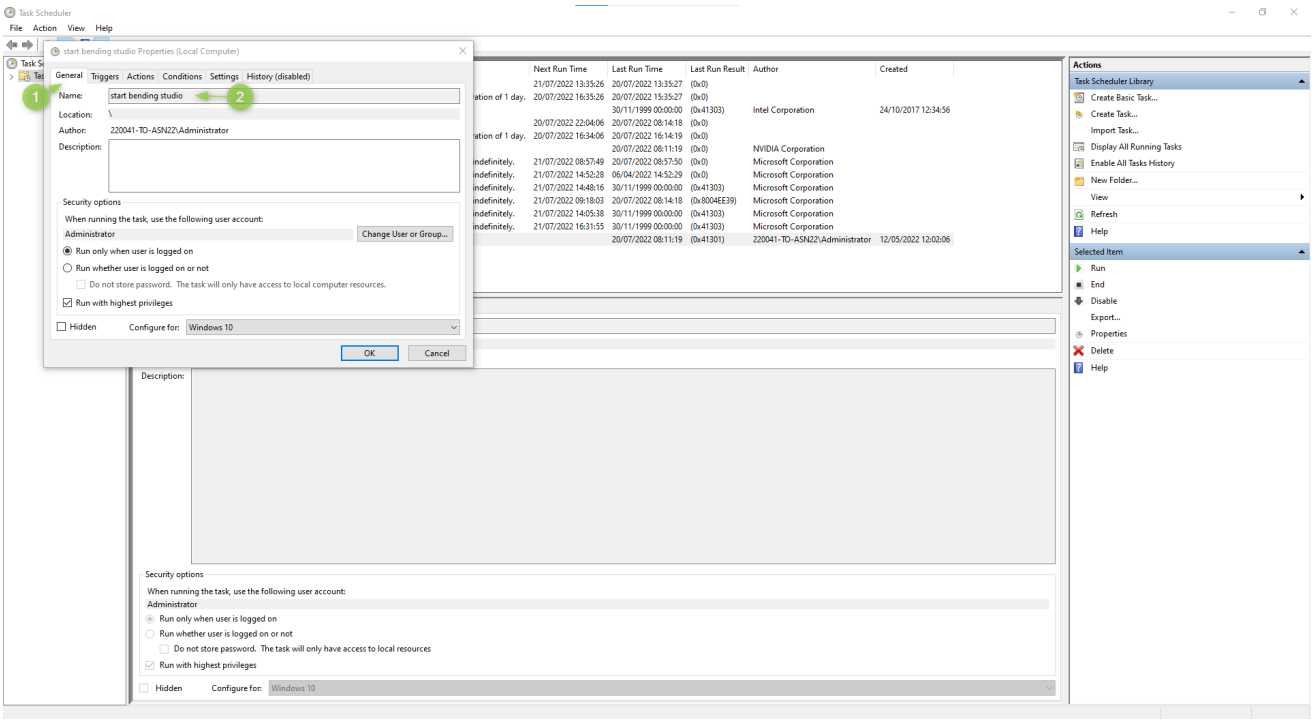
```
@echo off
cd C:\Program Files\AICON\BendingStudio
AppStudio.exe --loginName admin --loginPassword aicon1
              --startApp Automation --startAppArg StartServer
exit /b
```

2. Copy the script into a local folder on the installation disk, e.g. `C:\Aicon\BendingStudio`

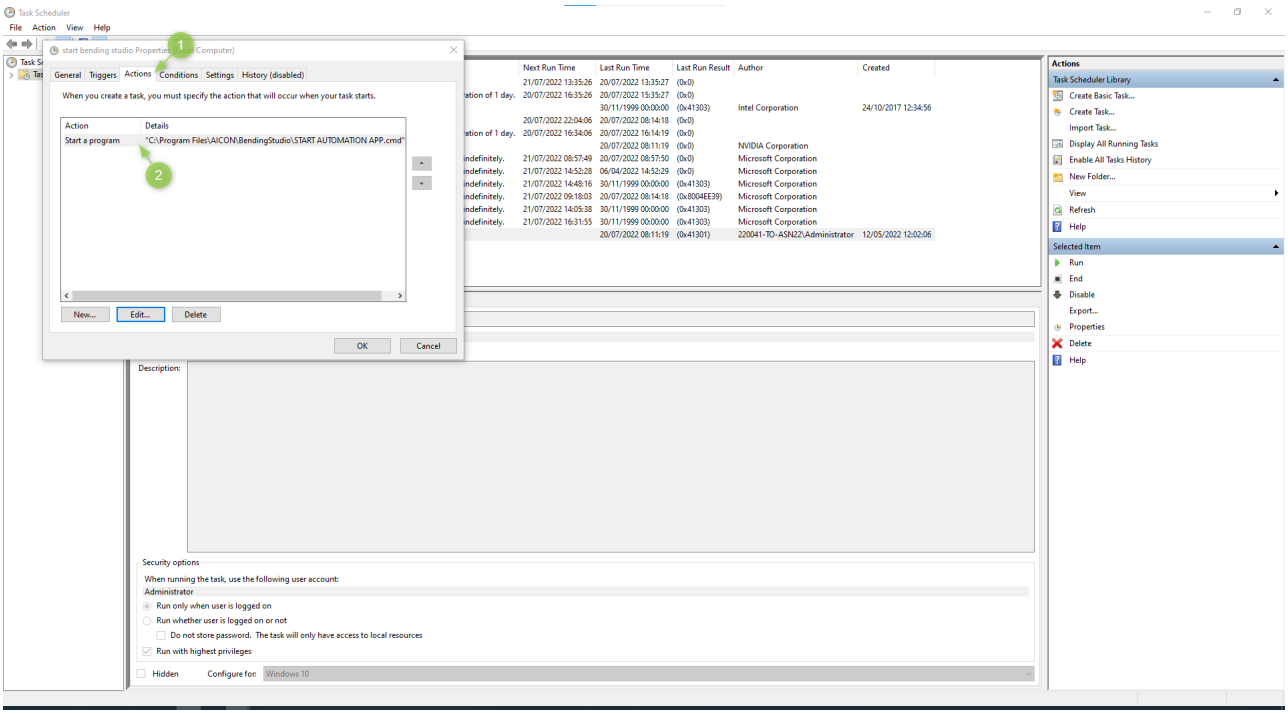


INFO Sample batch scripts can be found under:
`C:\BendingStudio\AutomationAPI\AutomationExamples\Batch`

3. Open Windows `Task scheduler` and give a name to the script.



4. Define the action `Start a program` for the batch file.



5. If these steps are configured, the BendingStudio XT software is started, logged in and the Automation application of BendingStudio XT is started automatically. After a successful execution, BendingStudio XT waits for automation commands.

